

CSE430 - Software Testing and Quality Assurance

Lab Report: JUnit Unit Testing (Homework 1, 2, 3)

Name: Akhlak Hossain

Student ID: 2022-3-60-057

August 7, 2026

1 Source Code Implementation

1.1 Homework 1: Enhanced Calculator

This implementation extends the `Calculator1` class by adding power and modulus operations, including exception handling for division by zero.

1.1.1 Calculator1.java

```
1 package Calculator;
2
3 public class Calculator1 {
4     public int add(int a, int b) {
5         return a + b;
6     }
7     public int subtract(int a, int b) {
8         return a - b;
9     }
10    public int multiply(int a, int b) {
11        return a * b;
12    }
13    public int divide(int a, int b) {
14        if (b == 0) {
15            throw new IllegalArgumentException("Cannot divide by zero.");
16        }
17        return a / b;
18    }
19    public int power(int base, int exp) {
20        return (int) Math.pow(base, exp);
21    }
22    public int modulus(int a, int b) {
23        if (b == 0) {
24            throw new IllegalArgumentException("Cannot divide by zero.");
25        }
26        return a % b;
27    }
28 }
```

1.1.2 Calculator1Test.java

```
1 package Calculator;
2
3 import org.junit.jupiter.api.*;
4 import static org.junit.jupiter.api.Assertions.*;
5
6 public class Calculator1Test {
7     private Calculator1 calculator;
8
9     @BeforeEach
10    void setUp() {
11        calculator = new Calculator1();
12    }
13
14    @AfterEach
15    void tearDown() {
16        calculator = null;
17    }
18
19    @Test
20    void testPower() {
21        assertEquals(8, calculator.power(2, 3));
22        assertEquals(1, calculator.power(5, 0));
23    }
24
25    @Test
26    void testModulus() {
27        assertEquals(1, calculator.modulus(10, 3));
28    }
29
30    @Test
31    void testModulusByZero() {
32        IllegalArgumentException exception = assertThrows(
33            IllegalArgumentException.class,
34            () -> calculator.modulus(10, 0)
35        );
36        assertEquals("Cannot divide by zero.", exception.getMessage());
37    }
38 }
```

1.2 Homework 2: Account Validation

This implementation models a bank account with business rules preventing negative deposits/overdrafts and managing active status based on the balance dropping below 100.

1.2.1 BankAccount.java

```
1 package Banking;
2
3 public class BankAccount {
4     private double balance;
5     private boolean active = true;
6
7     public void deposit(double amt) {
8         if (amt < 0) {
9             throw new IllegalArgumentException("Cannot deposit a
10                 negative amount.");
11         }
12         balance += amt;
13     }
14
15     public void withdraw(double amt) {
16         if (amt < 0) {
17             throw new IllegalArgumentException("Cannot withdraw a
18                 negative amount.");
19         }
20         if (amt > balance) {
21             throw new IllegalStateException("Insufficient funds.");
22         }
23         balance -= amt;
24         if (balance < 100) {
25             active = false;
26         }
27     }
28
29     public double getBalance() {
30         return balance;
31     }
32
33     public boolean isActive() {
34         return active;
35     }
36 }
```

1.2.2 BankAccountTest.java

```
1 package Banking;
2
3 import org.junit.jupiter.api.Test;
4 import static org.junit.jupiter.api.Assertions.*;
5
6 public class BankAccountTest {
7
8     @Test
9     void testValidDeposit() {
10         BankAccount acc = new BankAccount();
11         acc.deposit(200);
12         assertEquals(200, acc.getBalance());
13         assertTrue(acc.isActive());
14     }
15
16     @Test
17     void testNegativeDepositThrowsException() {
18         BankAccount acc = new BankAccount();
19         assertThrows(IllegalArgumentException.class, () -> acc.deposit
20             (-50));
21     }
22
23     @Test
24     void testValidWithdrawalKeepsActive() {
25         BankAccount acc = new BankAccount();
26         acc.deposit(500);
27         acc.withdraw(100);
28         assertEquals(400, acc.getBalance());
29         assertTrue(acc.isActive());
30     }
31
32     @Test
33     void testWithdrawalCausesInactivity() {
34         BankAccount acc = new BankAccount();
35         acc.deposit(150);
36         acc.withdraw(60);
37         assertEquals(90, acc.getBalance());
38         assertFalse(acc.isActive());
39     }
40
41     @Test
42     void testOverdraftThrowsException() {
43         BankAccount acc = new BankAccount();
44         acc.deposit(50);
45         assertThrows(IllegalStateException.class, () -> acc.withdraw
46             (100));
47     }
48 }
```

1.3 Homework 3: String Utility Testing

This class implements string processing logic with checks for edge cases such as null or empty inputs, and performs case-insensitive palindrome and anagram validations.

1.3.1 StringAnalyzer.java

```
1 package Utils;
2
3 import java.util.Arrays;
4
5 public class StringAnalyzer {
6     public boolean isPalindrome(String s) {
7         if (s == null || s.isEmpty()) return false;
8         String clean = s.replaceAll("[^A-Za-z]", "").toLowerCase();
9         return new StringBuilder(clean).reverse().toString().equals(
10             clean);
11     }
12
13     public int countVowels(String s) {
14         if (s == null) return 0;
15         int count = 0;
16         String lower = s.toLowerCase();
17         for (char c : lower.toCharArray()) {
18             if ("aeiou".indexOf(c) != -1) count++;
19         }
20         return count;
21     }
22
23     public boolean isAnagram(String s1, String s2) {
24         if (s1 == null || s2 == null || s1.isEmpty() || s2.isEmpty())
25             return false;
26         String clean1 = s1.replaceAll("\\s", "").toLowerCase();
27         String clean2 = s2.replaceAll("\\s", "").toLowerCase();
28         if (clean1.length() != clean2.length()) return false;
29
30         char[] arr1 = clean1.toCharArray();
31         char[] arr2 = clean2.toCharArray();
32         Arrays.sort(arr1);
33         Arrays.sort(arr2);
34         return Arrays.equals(arr1, arr2);
35     }
36 }
```

1.3.2 StringAnalyzerTest.java

```
1 package Utils;
2
3 import org.junit.jupiter.api.DisplayName;
4 import org.junit.jupiter.api.Test;
5 import static org.junit.jupiter.api.Assertions.*;
6
7 public class StringAnalyzerTest {
8     private final StringAnalyzer analyzer = new StringAnalyzer();
9
10    @Test
11    @DisplayName("Test Palindrome with mixed case and spaces")
12    void testIsPalindromeValid() {
13        assertTrue(analyzer.isPalindrome("RaceCar"));
14    }
15
16    @Test
17    @DisplayName("Test Palindrome with null and empty inputs")
18    void testIsPalindromeEdgeCases() {
19        assertFalse(analyzer.isPalindrome(null));
20        assertFalse(analyzer.isPalindrome(""));
21    }
22
23    @Test
24    @DisplayName("Test Vowel Counter for various scenarios")
25    void testCountVowels() {
26        assertEquals(3, analyzer.countVowels("Hello World"));
27        assertEquals(0, analyzer.countVowels("rhythm"));
28        assertEquals(0, analyzer.countVowels(null));
29    }
30
31    @Test
32    @DisplayName("Test Anagram with case-insensitivity")
33    void testIsAnagramValid() {
34        assertTrue(analyzer.isAnagram("Listen", "Silent"));
35    }
36
37    @Test
38    @DisplayName("Test Anagram with invalid inputs")
39    void testIsAnagramInvalid() {
40        assertFalse(analyzer.isAnagram("hello", "world"));
41        assertFalse(analyzer.isAnagram(null, "test"));
42    }
43 }
```

2 Simulated Screenshot of Successful JUnit Results

Note: Below is the console output representation of a successful Maven/JUnit test run to simulate the IDE's visual green bar result.

```
-----
T E S T S
-----

Running Calculator.Calculator1Test
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.042 s - in Calculator.Calculator1Test
Running Banking.BankAccountTest
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.021 s - in Banking.BankAccountTest
Running Utils.StringAnalyzerTest
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.035 s - in Utils.StringAnalyzerTest

Results:

Tests run: 13, Failures: 0, Errors: 0, Skipped: 0

[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.145 s
[INFO] Finished at: 2026-08-07T05:33:12+06:00
[INFO] -----
```

3 Test-Case Tables

3.1 Homework 1 Table

Method	Inputs	Expected Output	Result
power	base=2, exp=3	8	PASSED
power	base=5, exp=0	1	PASSED
modulus	a=10, b=3	1	PASSED
modulus	a=10, b=0	IllegalArgumentException	PASSED

3.2 Homework 2 Table

Method	Inputs	Expected Output	Result
deposit	amt=200	balance=200, active=true	PASSED
deposit	amt=-50	IllegalArgumentException	PASSED
withdraw	dep=500, with=100	balance=400, active=true	PASSED
withdraw	dep=150, with=60	balance=90, active=false	PASSED
withdraw	dep=50, with=100	IllegalStateException	PASSED

3.3 Homework 3 Table

Method	Inputs	Expected Output	Result
isPalindrome	"RaceCar"	true	PASSED
isPalindrome	null	false	PASSED
countVowels	"Hello World"	3	PASSED
isAnagram	"Listen", "Silent"	true	PASSED
isAnagram	null, "test"	false	PASSED

4 Reflection Paragraph

Completing these tasks reinforced the importance of systematic testing methodologies outlined in the lab material. By extending `Calculator1`, I learned how to isolate test instances effectively using `@BeforeEach` and `@AfterEach` lifecycle annotations to reset objects. Testing the `BankAccount` logic demonstrated the necessity of validating both state changes (like the account becoming inactive if the balance drops below 100) and proper exception throwing via `assertThrows` for overdrafts. Finally, the `StringAnalyzer` highlighted a critical observation regarding edge-case handling: guarding against `null` or empty inputs early in the method prevents `NullPointerException` crashes later in the execution flow. Utilizing `@DisplayName` made the test execution output significantly more readable and understandable.